

REGRESSION STUDY GUIDE

explanation1

Print-ready study guide with browser-rendered mathematics
and diagrams

Source: explanation1.md

Add geographic explanation of OLS and explanation of slope, tangent etc. Assume no knowledge of calculus, geometry and ML. and give me a pdf and md file Just for chapter01.md specially the text that i provide below

Good plan — let's cover the derivation and theory today (the conceptual heavy-lifting), and save the `fit()` build + stress-testing for tomorrow once you've had time to sit with this. That's a clean split point anyway.

Day 5 — Part 1: Deriving OLS with Calculus

Central idea: Day 4 reached $X^T X \hat{\beta} = X^T y$ through geometry (perpendicular projection). Today we reach the *exact same equation* through calculus. Geometry tells you **what** OLS does; calculus tells you **how** you'd actually locate that optimum by setting a derivative to zero. Both are true simultaneously — if your calculus ever contradicts the geometry, you've made an algebra error, not discovered new math.

5.1 — The one-parameter case (no intercept, warm-up)

$\hat{y}_i = \beta x_i$. The squared-error objective:

$$S(\beta) = \sum_i (y_i - \beta x_i)^2$$

Expand one term: $(y_i - \beta x_i)^2 = y_i^2 - 2\beta x_i y_i + \beta^2 x_i^2$. Sum over all i :

$$S(\beta) = \sum_i y_i^2 - 2\beta \sum_i x_i y_i + \beta^2 \sum_i x_i^2$$

This is just a **parabola in β** — a 1-variable quadratic, shaped like a bowl (since the β^2 coefficient $\sum x_i^2$ is positive). Differentiate and set to zero to find the bottom:

$$\frac{dS}{d\beta} = -2 \sum_i x_i y_i + 2\beta \sum_i x_i^2 = 0 \implies \hat{\beta} = \frac{\sum_i x_i y_i}{\sum_i x_i^2}$$

The second derivative $\frac{d^2 S}{d\beta^2} = 2 \sum_i x_i^2$ is positive whenever any $x_i \neq 0$ — confirming this stationary point is a genuine **minimum**, not a maximum or saddle.

5.2 — Adding the intercept (one feature, two parameters)

$\hat{y}_i = \beta_0 + \beta_1 x_i$, objective $S(\beta_0, \beta_1) = \sum_i (y_i - \beta_0 - \beta_1 x_i)^2$. Now there are **two unknowns**, so two partial derivatives, each set to zero.

Intercept partial: $\frac{\partial S}{\partial \beta_0} = -2 \sum_i (y_i - \beta_0 - \beta_1 x_i) = 0$. Divide by n and rearrange:

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

This single line proves something concrete and checkable: **the OLS line with an intercept always passes through the point $(\bar{x}, \bar{y})$** — the mean of the data. Not approximately — exactly, algebraically guaranteed.

Slope partial, after substituting the intercept relationship and simplifying:

$$\hat{\beta}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$$

Numerator: how much $\mathbf{x}$ and $\mathbf{y}$ move together (co-movement), centered around their means. Denominator: how much $\mathbf{x}$ varies on its own. **If every x_i is identical, the denominator is zero and the slope is undefined** — you can't estimate a slope from a feature that never changes. That's not a numerical quirk, it's the formula telling you the question ("how does $\mathbf{y}$ change as $\mathbf{x}$ changes?") is unanswerable when $\mathbf{x}$ never changes.

5.3–5.4 — The matrix version (the real generalization)

For p features at once: $S(\beta) = (\mathbf{y} - \mathbf{X}\beta)^T(\mathbf{y} - \mathbf{X}\beta)$. Expand it — treat this like expanding $(a - b)^2$ but with matrices, being careful about transposes:

$$S(\beta) = \mathbf{y}^T \mathbf{y} - \mathbf{y}^T \mathbf{X} \beta - \beta^T \mathbf{X}^T \mathbf{y} + \beta^T \mathbf{X}^T \mathbf{X} \beta$$

The middle two terms are each **scalars**, and a scalar equals its own transpose, so $\mathbf{y}^T \mathbf{X} \beta = (\mathbf{y}^T \mathbf{X} \beta)^T = \beta^T \mathbf{X}^T \mathbf{y}$ — they're the same number, just written two ways. Combine them:

$$S(\beta) = \mathbf{y}^T \mathbf{y} - 2\beta^T \mathbf{X}^T \mathbf{y} + \beta^T \mathbf{X}^T \mathbf{X} \beta$$

Now we need to differentiate this **with respect to the vector β** , which needs two rules, both provable by expanding into plain coordinates first (the book does this with a concrete 2×2 numeric example —

$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 3 & 4 \end{bmatrix}$, $\beta = (5, 6)$ — expanding $\beta^T \mathbf{A} \beta$ term-by-term as an ordinary polynomial, differentiating that polynomial normally, then checking the matrix shortcut $(\mathbf{A} + \mathbf{A}^T)\beta$ gives the identical answer, **(44, 68)**, either way):

1. **Linear term:** $\nabla_{\beta}(\beta^T \mathbf{c}) = \mathbf{c}$ for any constant vector $\mathbf{c}$. So $\nabla_{\beta}(-2\beta^T \mathbf{X}^T \mathbf{y}) = -2\mathbf{X}^T \mathbf{y}$.
2. **Quadratic term:** $\nabla_{\beta}(\beta^T \mathbf{A} \beta) = (\mathbf{A} + \mathbf{A}^T)\beta$ in general; since $\mathbf{A} = \mathbf{X}^T \mathbf{X}$ is **symmetric** ($\mathbf{A} = \mathbf{A}^T$), this simplifies to $\nabla_{\beta}(\beta^T \mathbf{X}^T \mathbf{X} \beta) = 2\mathbf{X}^T \mathbf{X} \beta$.

Combine everything (the $\mathbf{y}^T \mathbf{y}$ term vanishes — it has no β in it at all):

$$\nabla_{\beta} S(\beta) = -2\mathbf{X}^T \mathbf{y} + 2\mathbf{X}^T \mathbf{X} \beta$$

Set the gradient to zero — this is the vector generalization of "set the derivative to zero" — and you land, again, exactly on:

$$\mathbf{X}^T \mathbf{X} \hat{\beta} = \mathbf{X}^T \mathbf{y}$$

Same equation as Day 4's geometry, reached by an entirely different route. That convergence is not a coincidence — it's confirmation that both derivations describe the same true minimum.

5.5 — When can you actually solve for $\hat{\beta}$?

If $X^T X$ is **invertible**: $\hat{\beta} = (X^T X)^{-1} X^T y$. This closed form only exists when X has **full column rank** — plain language: no column of X can be built exactly from the others, and there's enough independent information to pin down every parameter.

Worth knowing *why* this is guaranteed to be a minimum, not just a stationary point: the **Hessian** (matrix of second derivatives) is $\nabla_{\beta}^2 S(\beta) = 2X^T X$. For any vector z : $z^T X^T X z = (Xz)^T (Xz) = \|Xz\|_2^2 \geq 0$ — a squared length can never be negative. So $X^T X$ is **positive semidefinite**, meaning $S(\beta)$ is convex — bowl-shaped, no local minima that aren't also the global minimum. If X has full column rank, $Xz \neq 0$ for every nonzero z , making this strictly positive, which makes the minimizer **unique**.

5.6 — Why real code never actually computes $(X^T X)^{-1}$

Mathematically valid, computationally risky: **forming $X^T X$ squares the condition number** of the problem, amplifying floating-point error. Two better options, in order of preference:

```
beta_hat = np.linalg.solve(X.T @ X, X.T @ y)          # solve the system directly, don't invert
beta_hat, _, rank, singular_values = np.linalg.lstsq(X, y, rcond=None) # best: built for least-squares
```

`lstsq` also hands you `rank` and `singular_values` for free — which turns out to matter a lot in 5.7.

A layered warning worth internalizing now, because it'll matter for the rest of the book: solving OLS correctly is purely an *algebra* question. It does **not** require residuals to be normally distributed, doesn't require anything about the true error term — just that X has full column rank. Three separate layers exist and must not be conflated:

1. **Algebra** — did the program minimize SSR correctly?
2. **Statistical generalization** — does this sample support claims about a wider population?
3. **Causal interpretation** — does the design actually identify an intervention's effect (this is all of Chapter 6)?

Getting layer 1 perfect says *nothing* about layers 2 or 3 — echo of Day 1's whole point about prediction/explanation/causation needing different defenses.

5.7 — Rank failure in practice: perfect multicollinearity

If X contains both cable length in km **and** the same length in metres ($x_m = 1000x_{km}$), one column is an exact multiple of another. The model literally cannot decide how to split the coefficient between them — infinitely many (β_{km}, β_m) pairs produce the identical combined prediction. `np.linalg.solve` on the normal equations will **fail outright** (singular matrix). `lstsq` won't crash — it'll return *a* solution — but the individual coefficients on km and m are **not separately meaningful**; you'd be reading noise as if it were signal. The fix is to remove the redundant column or redesign the features, not to treat "the code ran" as success.

That's the full derivation — scalar case, intercept case, matrix case, positive-semidefiniteness, numerical practice, and the rank-failure trap. This is dense; sit with it overnight as planned. Tomorrow we'll build `fit()` and run the stress tests.

Before you go, a check on today's material only — five questions, explanation-matched, no coding required yet:

1. In 5.2, the formula $\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$ proves the fitted line passes through $(\bar{x}, \bar{y})$. Why does the *denominator* of $\hat{\beta}_1$ becoming zero (all x_i identical) make the slope literally impossible to compute, not just numerically unstable?
2. In the matrix expansion (5.3), why are $y^T X \beta$ and $\beta^T X^T y$ guaranteed to be equal? State the actual rule being used, not just "they're the same."
3. What does it mean, precisely, for $X^T X$ to be "positive semidefinite," and why does that property specifically guarantee $S(\beta)$ has no local minimum that isn't also the global one?
4. In 5.7's multicollinearity example, `lstsq` still returns numbers. Explain why "the code produced coefficients" is not the same as "the coefficients are meaningful" — connect this back to Day 1's estimand idea if you can.
5. Restate, in your own words, why solving OLS correctly (layer 1: algebra) tells you nothing about whether you can trust the coefficients as an explanation of *why* costs go up (layer 3-adjacent territory, though technically layer 2/3 both apply).